

TEXT-TO-SPEECH

Classical TTS: Text Frontend, Acoustic Models & Vocoders

From text normalization to perceptual evaluation

Course Overview

Part A · The TTS pipeline

- Text normalization & G2P
- Acoustic models (Tacotron 2 & FastSpeech)
- Prosody, style & speaker conditioning
- Alignment mechanisms & Duration modeling
- Intermediate representations (Mel-spectrograms)

Part B · The vocoder & evaluation

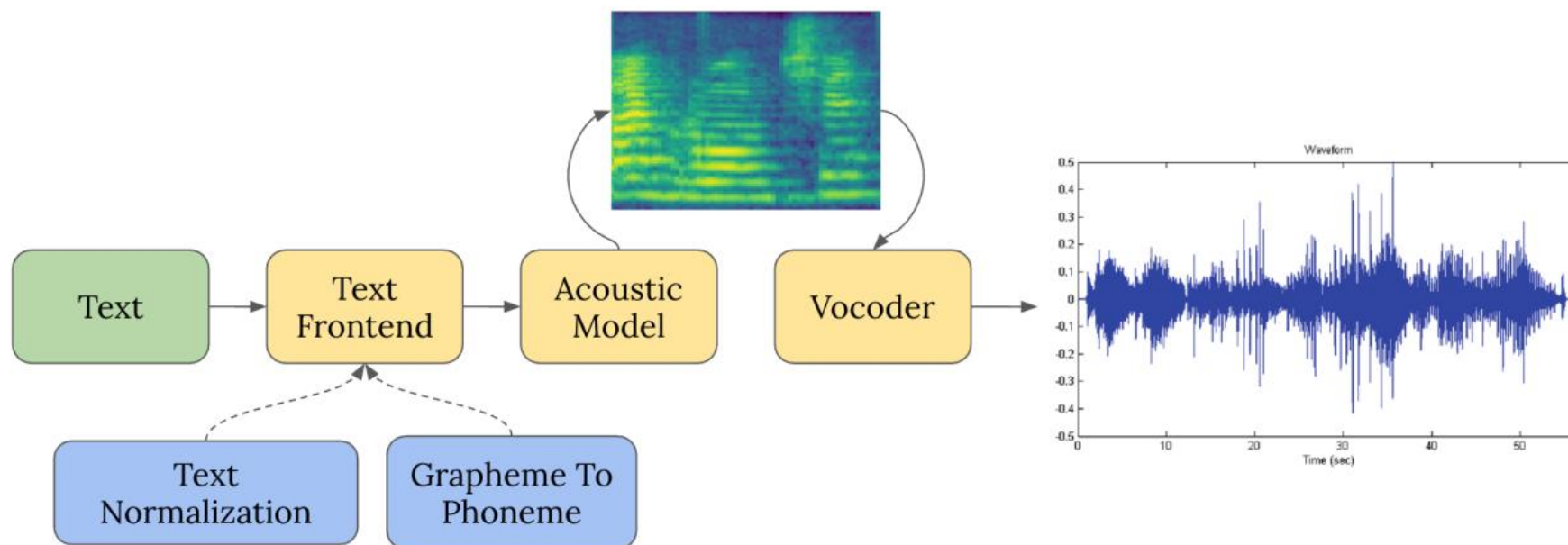
- Neural vocoders (WaveNet → Parallel WaveGAN → MelGAN → HiFi-GAN)
- Waveform synthesis from spectral features
- Subjective evaluation (MOS, MUSHRA, preference tests)
- Objective metrics (Confidence intervals)

Learning objectives

- Trace the full TTS pipeline from text normalization down to waveform generation.
- Distinguish training vs. inference modes and implicit vs. explicit alignment mechanisms.
- Connect acoustic model & neural-vocoder architectures to their respective loss functions.
- Select speech quality evaluation methods (MOS/MUSHRA) and interpret statistical confidence intervals.

Prerequisites: STFT, convolution, attention and basic statistics

What is TTS?



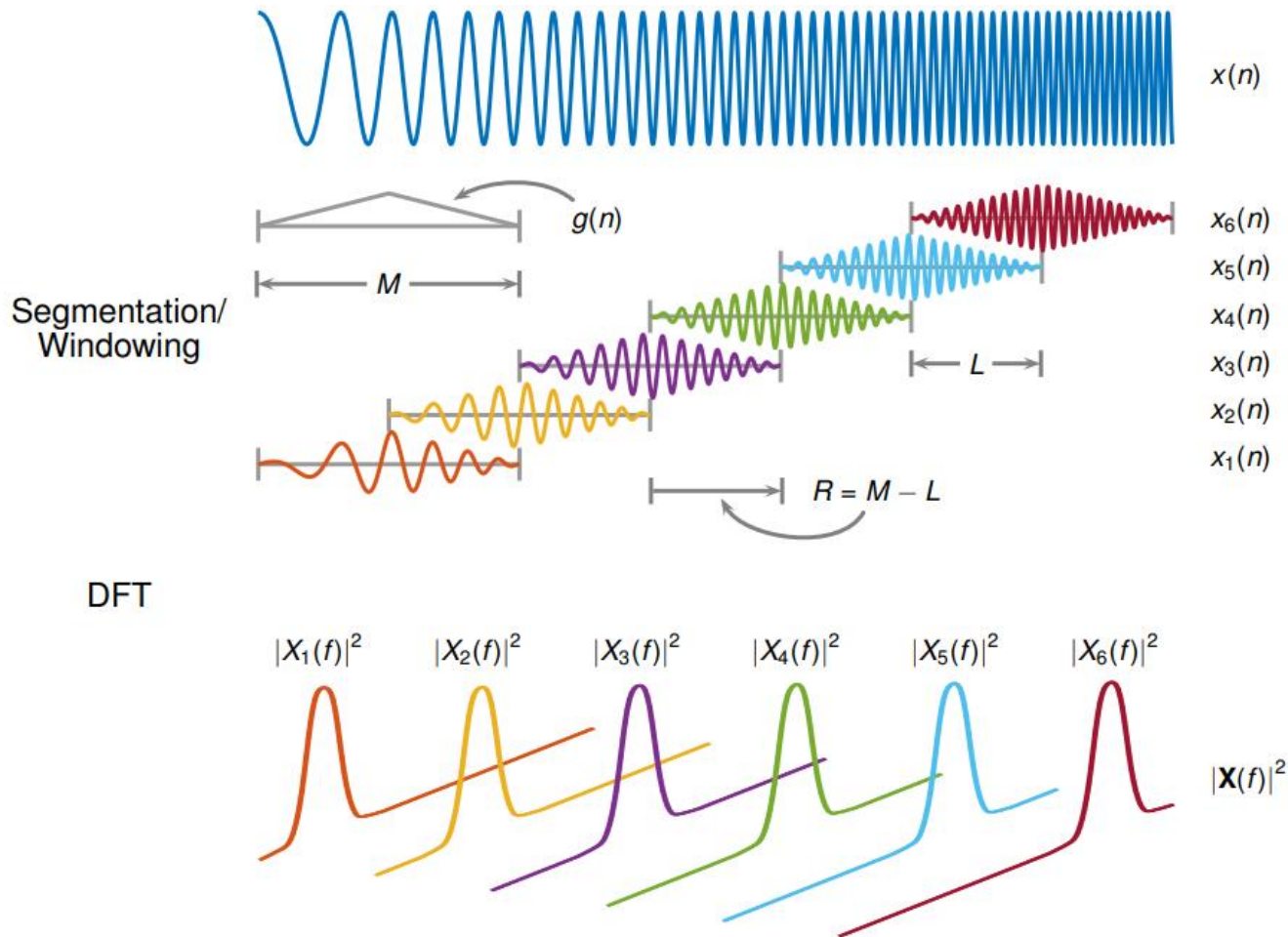
Three Modules

- **Text frontend** turns ambiguous writing into a pronounceable sequence.
- **Acoustic model** predicts a smooth, time-frequency target.
- **Vocoder** restores phase, fine structure, and audible detail.

Why stage the problem?

- Different modules operate at **different time scales**.
- **Intermediate targets** are easier to supervise than raw waveform samples.
- Each module can be replaced, profiled, and debugged **independently**.

Mel-spectrograms trade exactness for trainability



Pipeline

Step 1: Framing & windowing

Segment raw waveform $x(n)$ into overlapping frames using a window function $g(n)$ (e.g., Hann window).

Step 2: Apply DFT (STFT)

Apply DFT to each frame to compute the magnitude / power spectrum $|X(f)|^2$.

Step 3: Mel Filterbank Pooling

Pass magnitude spectra through non-linear triangular filterbanks to extract energy in Mel scale.

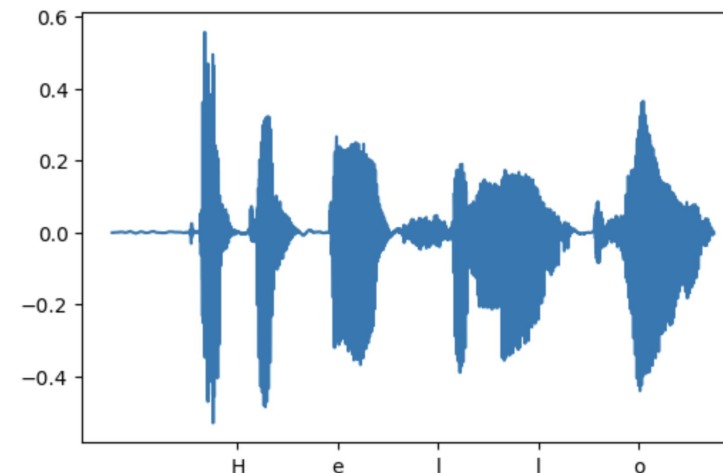
Step 4: Log compression & stacking

Apply dynamic-range compression and stack frame vectors into a 2D mel-spectrogram matrix.

Why Mel-spectrogram as an Intermediate Target?

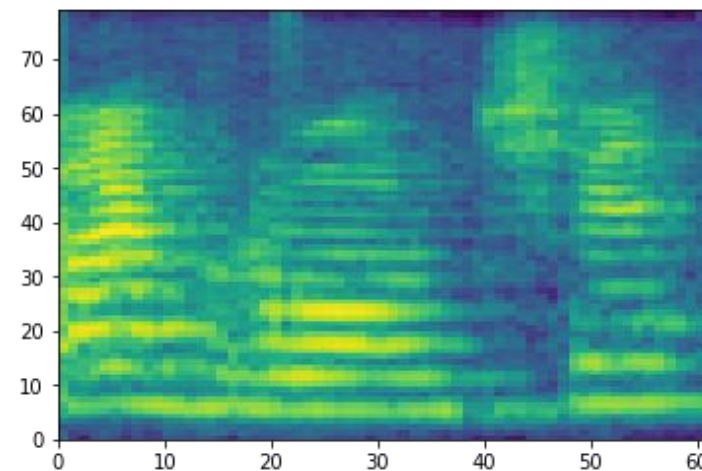
Waveform (Raw Time-Domain Audio)

- **High temporal resolution:** 24k-48k samples per seconds (24 kHz+)
- **Phase sensitivity:** Minor phase/timing shifts produce huge point-wise errors, yet sound virtually identical to human ears.
- **Loss mismatch:** Point-wise losses (e.g., L1/L2) severely penalize perceptually equivalent waveforms.

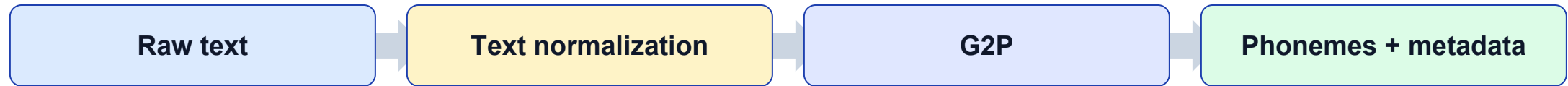


Mel-spectrogram

- **Lower frame rate:** Reduced temporal dimension (~80–100 frames/sec), making sequence modeling manageable.
- **Perceptual alignment:** Mel scale mimics the non-linear frequency resolution of human auditory perception.
- **Effective objective:** MSE in log-mel space provides a stable and perceptually meaningful regression target.



Text frontend: Text Normalization + G2P



1. Text normalization

Converting non-standard text into canonical spoken words.

e.g.

“10 kg”

“ten kilograms”

“\$12”



“twelve dollars”

“21/09”

“twenty-first of September”
(Context-depended)

2. Grapheme-to-Phoneme (G2P)

Mapping written letters (grapheme) to acoustic sound units (phonemes).

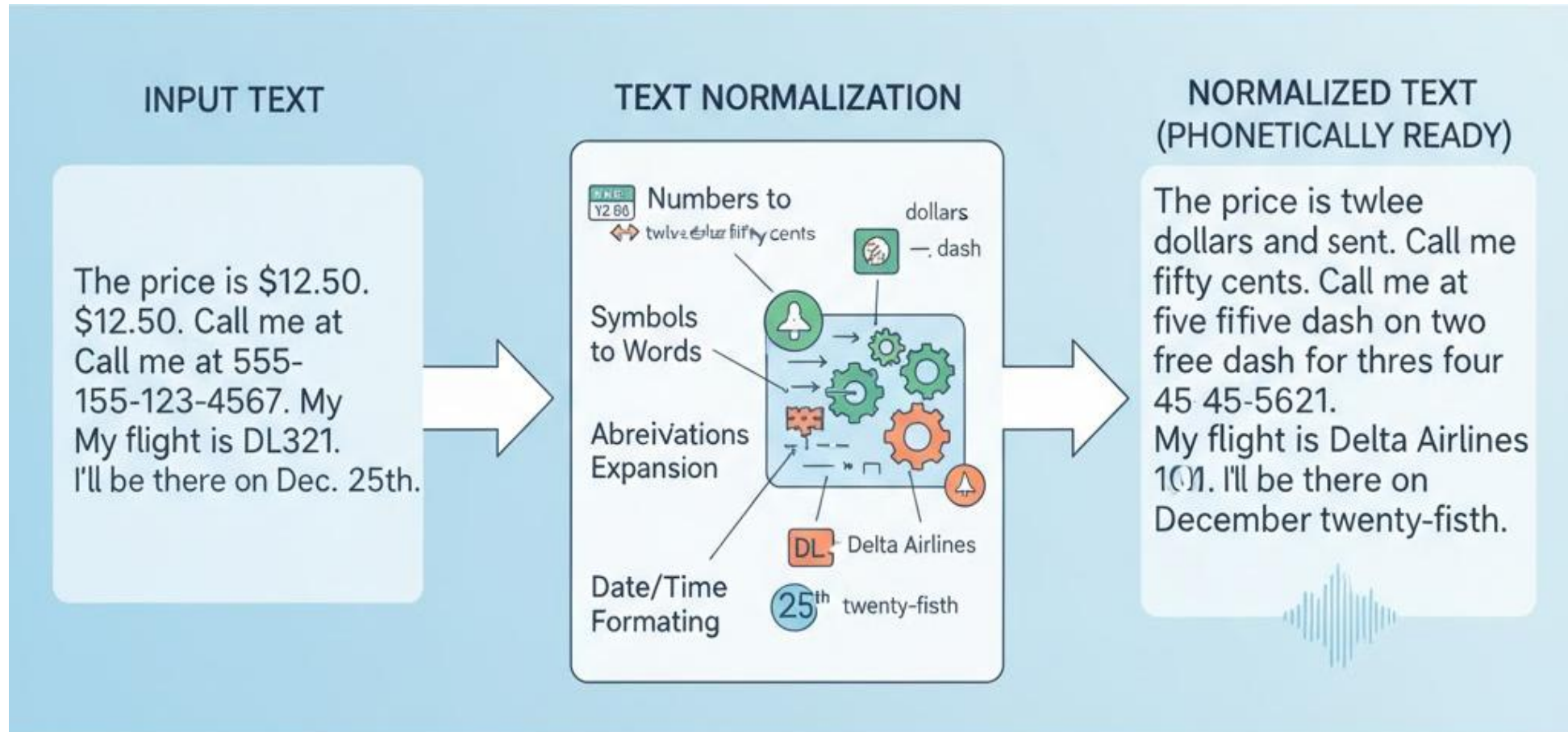
e.g.

“read”



“/r i: d/” or “/r ɛ d/”

Text frontend: Text normalization



Text frontend: Text normalization Strategies

1. Rules-base & finite-state transducers(FST)

- Relying on linguistics rules written by humans.

For instance, it knows that the number following the "\$" is usually pronounced as "dollars".

2. Seq2Seq (Neural Networks)

- Based on learning the transformation rules of the text from the model

Input is an unstandardized text sequence, while output is the standardized text sequence.

3. Hybrid systems

- Use **Rule-based** approach to handle common situations, and leave the unmatched parts to **seq2seq**.

Text frontend: G2P Strategies

1. Lexicon Lookup

- Look up the word in the dictionary.

For instance, [CMU Pronouncing Dictionary](#).

2. Rules-base & finite-state transducers(FST)

- Utilizing the pronunciation rules summarized by linguists

For instance, In English, words ending with "tion" usually have the pronunciation /ʃən/.

3. Seq2Seq

- Use **Rule-based** approach to handle common situations, and leave the unmatched parts to **seq2seq**.

The input is a sequence of letters, and the output is a sequence of phonetic symbols.

	monophthongs				diphthongs			
	i:	ɪ	ʊ	u:	ɪə	eɪ		
	sheep	ship	good	shoot	here	wait		
	e	ə	ɜ:	ɔ:	ʊə	ɔɪ	əʊ	
VOWELS	bed	teacher	bird	door	tourist	boy	show	
	æ	ʌ	ɑ:	ɒ	eə	aɪ	aʊ	
	cat	up	far	on	hair	my	cow	
CONSONANTS	p	b	t	d	tʃ	dʒ	k	g
	pea	boat	tea	dog	cheese	June	car	go
	f	v	θ	ð	s	z	ʃ	ʒ
	fly	video	think	this	see	zoo	shall	television
	m	n	ŋ	h	l	r	w	j
	man	now	sing	hat	love	red	wet	yes

Review: Frontend Decisions

Meaning or pronunciation?

1. Why can 21.09 have more than one reading?
2. Which module resolves an unseen name's pronunciation?
3. Why cannot a high-quality vocoder fix an incorrectly expanded number?

Reference answers: Frontend Decisions

Correct synthesis starts with correct interpretation

1. Locale, domain and sentence context distinguish a date from a number.

Date: "twenty-first of September" or "September twenty-first"

Number: "twenty-one point zero nine"

2. **G2P (Grapheme-to-Phoneme).**

It uses a custom lexicon override for hardcoded entries or a neural/learned G2P model as a fallback for OOV (out-of-vocabulary) words.

3. The vocoder follows acoustic conditioning.

It cannot recover the intended written meaning from incorrectly predicted features.

Acoustic Model: Tacotron 2

1. Text Encoder

Encoder character/phoneme sequence into rich contextual representation via **3 Conv layer + LSTM**.

2. Location-sensitive attention

Aligns text positions with decoder steps, **using cumulative attention weights** to prevent repeating or skipping words.

3. MelSpectrogram Decoder

Autoregressively predicts Mel-spectrogram frames sequentially using a **2-layer LSTM** network.

4. Pre-Net & Post-Net

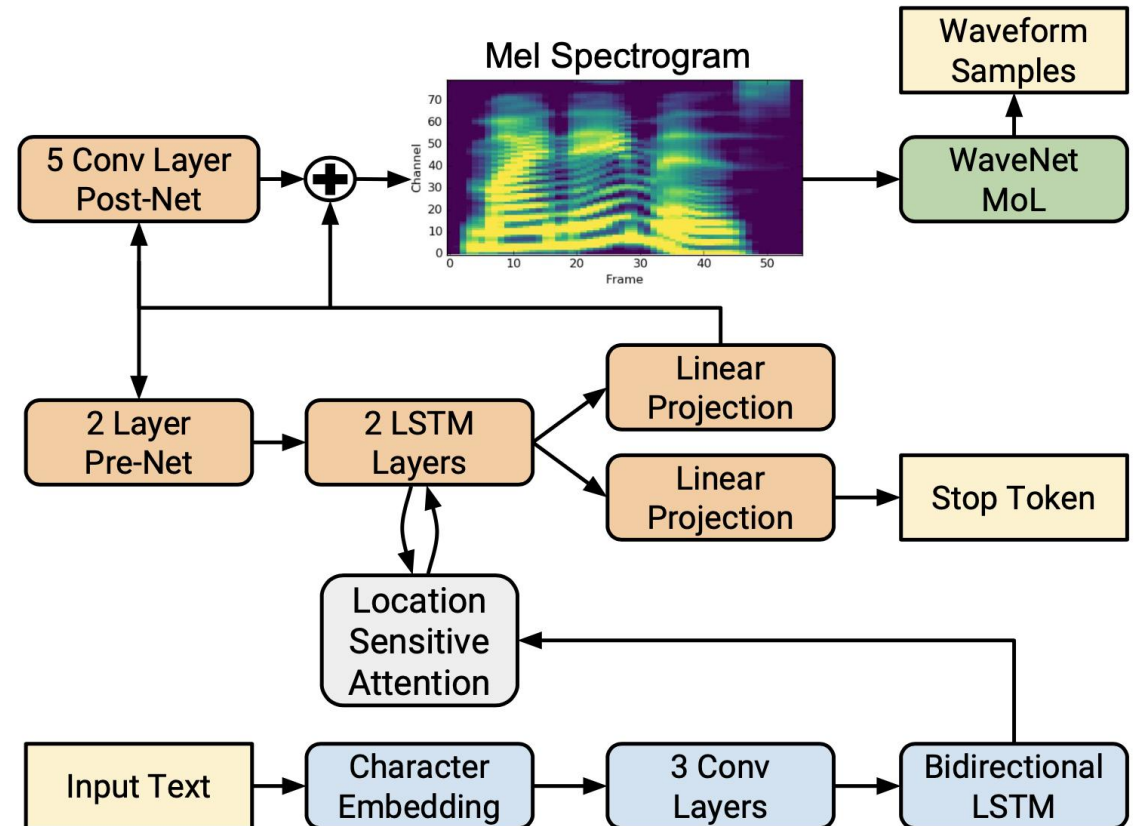
Pre-Net: Acts as an information bottleneck to prevent autoregressive memory overfitting.

Post-Net: 5-layer CNN that predicts residual features to refine smooth spectrograms.

5. Loss Functions (MSE & CE)

MSE: Applied before and after Post-Net for Mel-spectrogram reconstruction loss.

CE: Predicts the probability of the Stop Token for dynamic sequence termination.



Tacotron 2: Location-sensitive attention (LSA)

Mathematical Formulation

$$e_{i,j} = w^\top \tanh(Ws_{i-1} + Vh_j + Uf_{i,j} + b)$$

$$e_{i,j} = \text{Score}(s_{i-1}, \alpha_{i-1,j}, h_j)$$

$$\alpha_{i,j} = \exp(e_{i,j}) / \sum_{j=1}^L \exp(e_{i,j})$$

$$f_i = F * \alpha_{i-1}$$

- s_{i-1} : Previous decoder state (Acoustic context)
- h_j : Encoder output at position j (Text content)
- $f_{i,j}$: Accumulated location feature from 1D conv F

Mechanism & Failure Modes

Dual-Driven Attention Mechanism

- **Content Match**: which phoneme matches this frame?
- **Location Aware**: which positions have already received attention?
- **Synergy**: Enforces monotonic progression and prevents random attention jumping.

Failure signature

- **Word Repetitions**: Model gets stuck in a loop at position j .
- **Word Skips**: Attention jumps forward prematurely.
- **Endless Generation**: Model fails to trigger the Stop Token.

Tacotron 2: LSA Code Implementation

$$e_{i,j} = w^{\top} \tanh(W s_{i-1} + V h_j + U f_{i,j} + b)$$

NVIDIA/tacotron2 / model.py

185cd24e04 · L57-63

```
57 processed_query = self.query_layer(query.unsqueeze(1))
58 processed_attention_weights = self.location_layer(attention_weights_cat)
59 energies = self.v(torch.tanh(
60     processed_query + processed_attention_weights + processed_memory))
61
62 energies = energies.squeeze(-1)
63 return energies
```

One score per
text position

Decoder query,
location features and
encoder memory
combine before scoring.

Energies: [B, N]
Softmax follows in
the calling path.

Unchanged source excerpt · visual wrapping and common indentation offset only

Location sensitivity supplies position information, not a hard monotonic constraint.

NVIDIA Corporation · NVIDIA/tacotron2 · BSD-3-Clause · model.py L57-63 · commit 185cd24e04

Tacotron 2: Pre-Net & Post-Net

Pre-Net (Information Bottleneck)

Structure: 2 FC layers (256 units) with ReLU & Dropout ($p=0.5$).

Input: Previous Mel-spectrogram frame.

Core Role:

- Acts as an **information bottleneck** to prevent the autoregressive decoder from merely memorizing the previous frame.
- Reduces over-reliance on historical speech frames, forcing attention to rely on text representations.

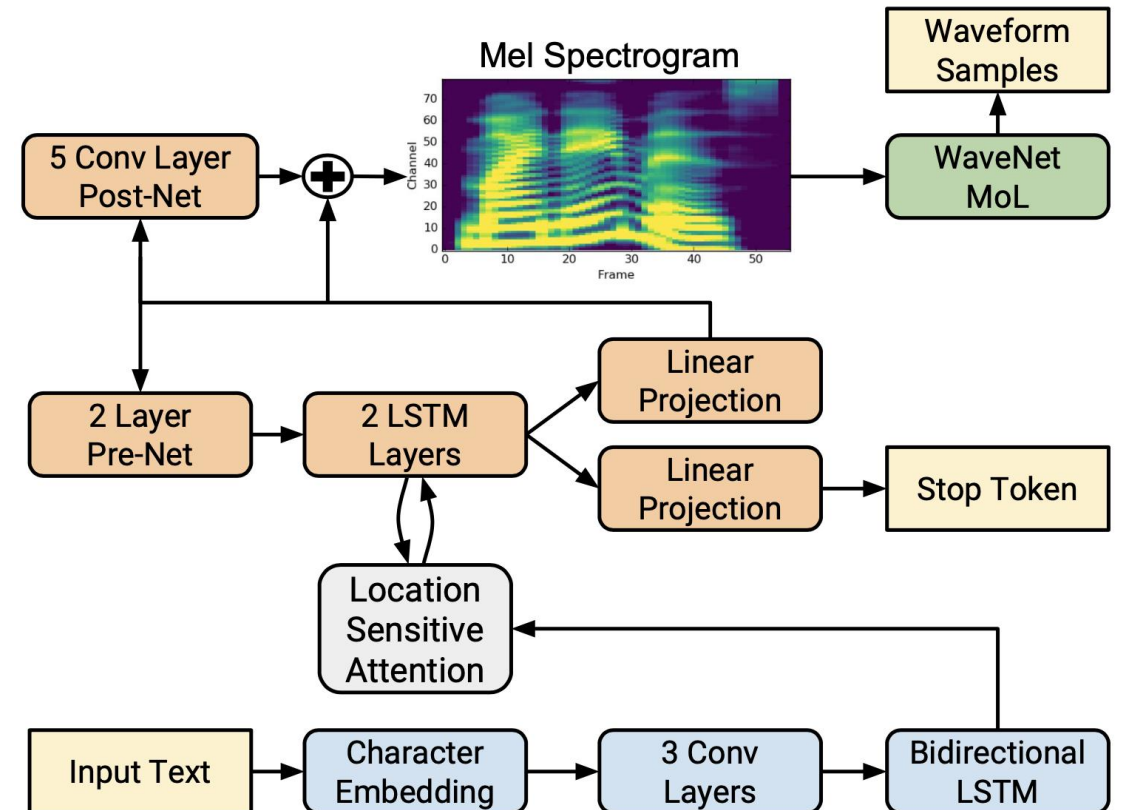
Post-Net (Spectrogram Refinement)

Structure: 5 1D Conv layers (Batch Normalization + Tanh).

Input: Coarse Mel-spectrogram from linear projection.

Core Role:

- Predicts a **residual signal** to add to the coarse
Mel prediction: $Mel_{final} = Mel_{coarse} + Post-Net(Mel_{coarse})$
- Sharpens local time-frequency details and smooths synthesis artifacts.
- Supervision:** Both coarse and refined Mels are optimized using **MSE Loss**.



Tacotron 2 : Acoustic Fidelity & Termination

Mel reconstruction(Acoustic Fidelity)

$$L_{mel} = || M_{pred} - M_{target} ||^2$$

- **Frame-level regression**

Directly minimizes the distance between predicted and ground-truth Mel-spectrograms.

- **Why Mel-Spectrogram?**

Smoother, lower-dimensional intermediate representation compared to raw time-domain waveforms.

Stop prediction(Sequence Termination)

$$L_{stop} = \text{CE}(\text{stop}_{pred}, \text{stop}_{target})$$

- **Binary Classification Task**

Predicts the probability [0 , 1] that the current frame is the end of the sequence.

- **Inference Threshold Control**

Stops generation when $p(\text{stop}) > \text{threshold}$ (typically 0.5).

- **Calibration Risks**

Threshold too low: Premature speech truncation (words cut off at the end).

Threshold too high: Trailing silence or infinite generation loops with static noise.

The loss does not guarantee natural speech: it only defines what the model is asked to match.

Code: stopping at inference

NVIDIA/tacotron2 / model.py

185cd24e04 · L435-449

```
435 while True:
436     decoder_input = self.prenet(decoder_input)
437     mel_output, gate_output, alignment = self.decode(decoder_input)
438
439     mel_outputs += [mel_output.squeeze(1)]
440     gate_outputs += [gate_output]
441     alignments += [alignment]
442
443     if torch.sigmoid(gate_output.data) > self.gate_threshold:
444         break
445     elif len(mel_outputs) == self.max_decoder_steps:
446         print("Warning! Reached max decoder steps")
447         break
448
449     decoder_input = mel_output
```

Unchanged source excerpt · visual wrapping and common indentation offset only

Prediction feeds back; sigmoid gate or max steps ends the loop. This excerpt assumes one utterance.

NVIDIA Corporation · NVIDIA/tacotron2 · BSD-3-Clause · model.py L435-449 · commit 185cd24e04

Tacotron 2 : Guided attention

1. Motivation: The Alignment Bottleneck

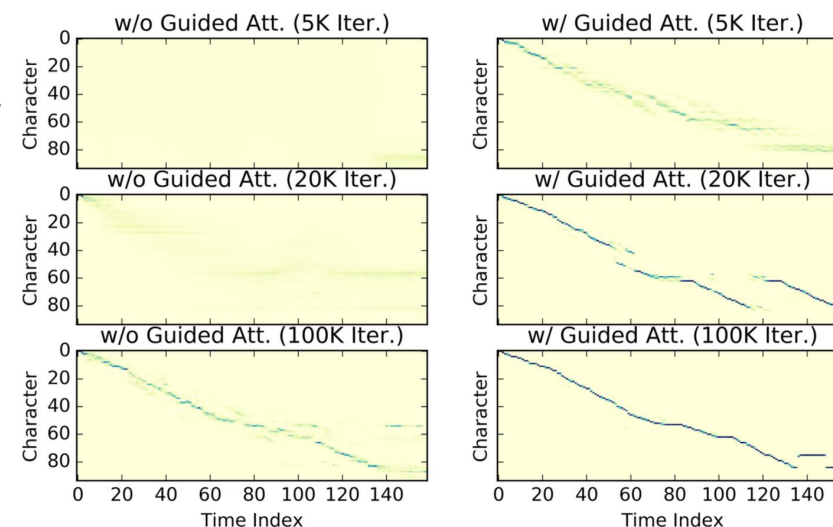
Problem: Pure end-to-end attention takes tens of thousands of steps to learn monotonic alignment from scratch, often causing word skipping or repeating.

Solution: Introduce a soft diagonal prior L_{att} to guide attention learning during early training stages.

2. Mathematical Formulation & Intuition

$$\mathcal{L}_{\text{att}}(A) = \mathbb{E}_{nt}[A_{nt}W_{nt}], \text{ where } W_{nt} = 1 - \exp\left\{-\left(n/N - t/T\right)^2/2g^2\right\}$$

- Normalized text position and acoustic time progress roughly together.
- Penalty Matrix W_{nt} :**
 - On-diagonal ($n/N \approx t/T$): $W_{nt} \rightarrow 0 \rightarrow$ No penalty.
 - Off-diagonal (Far from diagonal): $W_{nt} \rightarrow 1 \rightarrow$ High penalty.
- Bandwidth Hyperparameter g :** Controls the flexibility allowance for pauses and varying speech rates.



Tacotron 2 : Global Style Tokens (GST) - Training Phase

The one-to-many problem

A single text input corresponds to multiple valid audio outputs with varying pitch, pace, energy, and emotion. Text alone lacks acoustic variation details.

GST Core Training Workflow

1. Reference Encoder

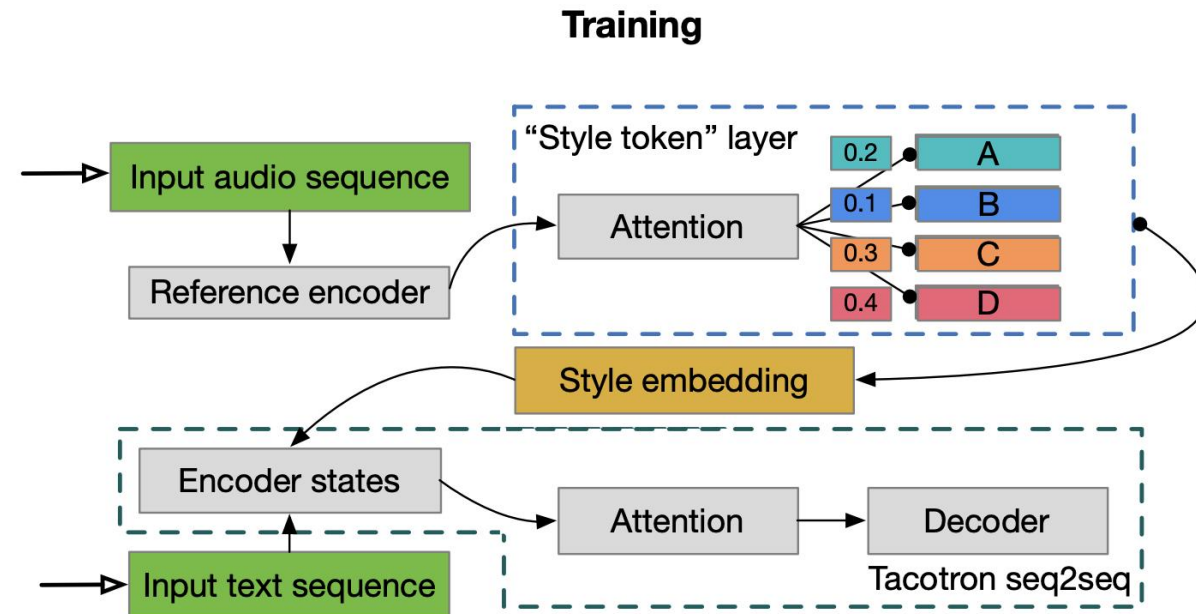
- Extracts a fixed-length **Reference Embedding** from variable-length target audio.

2. Style Token Layer

- Maintains a bank of K learnable embeddings (Tokens A, B, C...).
- Uses Multi-Head Attention to calculate similarity weights between Ref Embedding and Tokens.

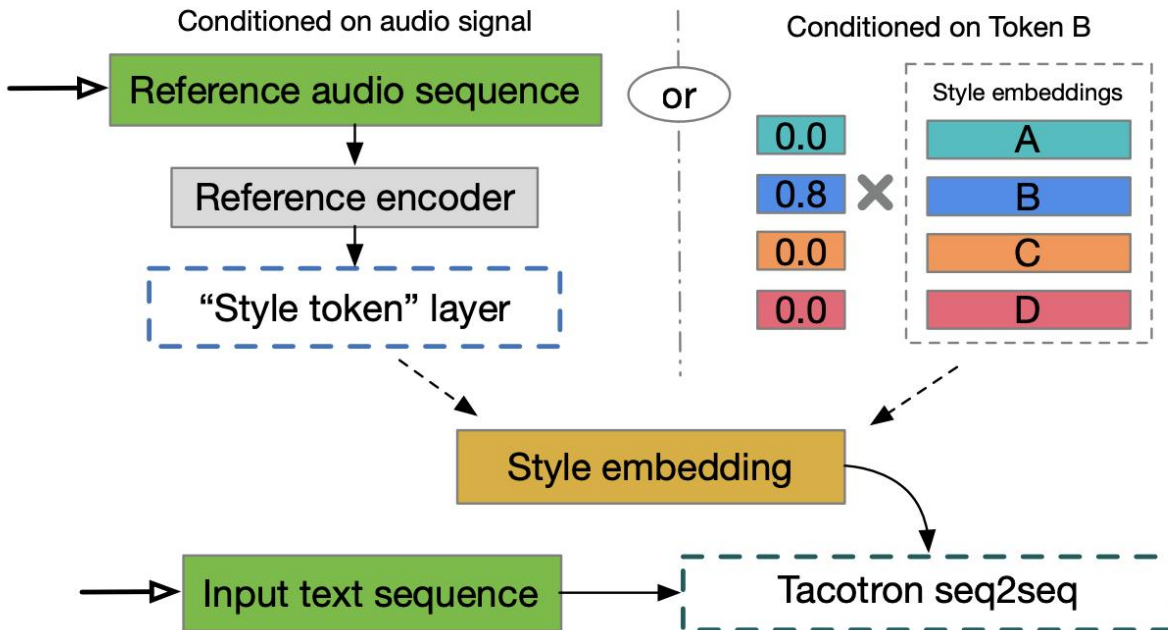
3. Conditioning the Decoder

- Multiplies weights with tokens to form the final Style Embedding.
- Concatenated/added to text encoder outputs to guide speech generation.



Tacotron 2 : Global Style Tokens (GST) - Inference & Limitations

Inference



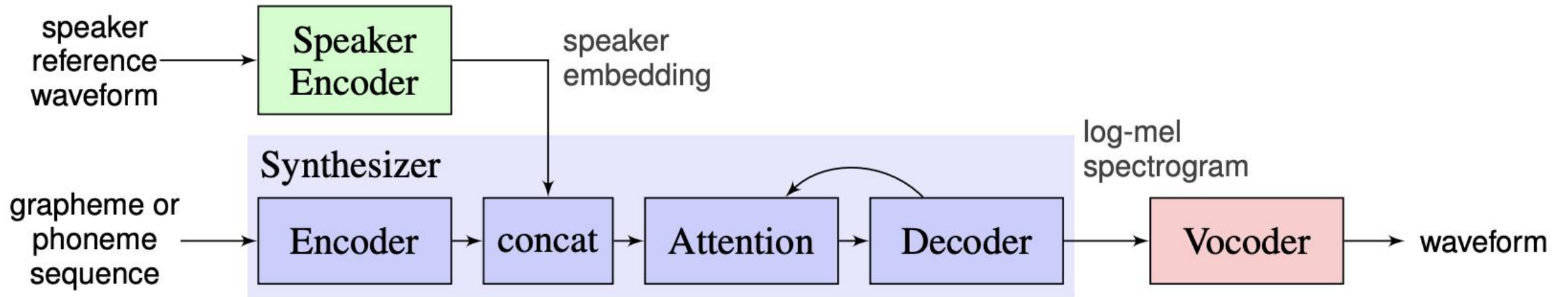
Two control modes

- **Reference audio:** transfer a style without a label.
- **Token weights:** explore or reproduce a discovered style.
- Both paths produce a style embedding that enters the acoustic model.

Caveat

Unsupervised tokens are not guaranteed to map cleanly to human labels such as 'happy' or 'sad'. Interpretability is empirical.

Multispeaker TTS



Speaker encoder (SE)

- Trained on a speaker verification task.
- Map the voices of different people onto vectors that are vastly different from each other.

Text-independent Task

- Identity should survive content changes.
- Extract the common and stable vocal characteristics that belong to the speaker.

Review and answers: acoustic modeling

1. Does location sensitivity guarantee monotonic alignment?
2. What previous frame is used at inference?
3. Are GST tokens fixed emotion labels?

Review and answers: acoustic modeling

1. Does location sensitivity guarantee monotonic alignment?

No. It supplies position-aware features, not a hard constraint.

2. What previous frame is used at inference?

The generated frame, after the initial input.

3. Are GST tokens fixed emotion labels?

No. Their meanings are learned and need inspection.

Acoustic Model : FastSpeech (Non-Autoregressive TTS)

Tacotron 2

- **Frame-by-frame** decoding
- **Implicit** Attention alignment
- Slower, vulnerable to skipping/repetition



FastSpeech

- **Parallel** Mel Generation
- **Explicit** Duration-based Alignment
- Extremely fast ($\sim 30\times$), robust & controllable

Key Architectural Components

1. Feed-Forward Transformer (FFT) Block

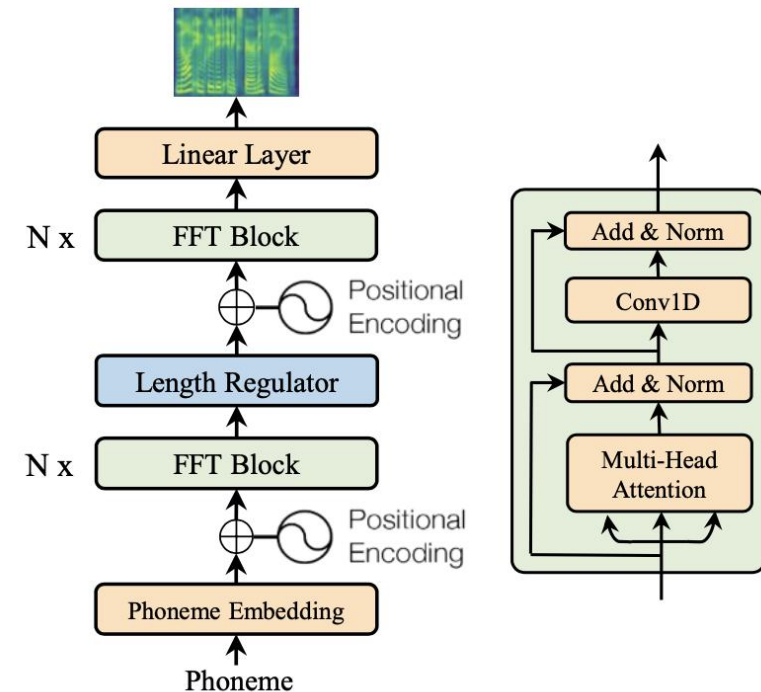
- Replaces RNNs with 1D Convolutions + Multi-Head Self-Attention for massive parallelism

2. Duration Predictor

- Predicts frame-level duration for each phoneme (trained on Teacher attention weights)

3. Length Regulator (The Core Innovation)

- Expands phoneme sequence (N) to match audio frame length (T) based on duration predictions.



FastSpeech : Duration Predictor & Length Regulator

Duration predictor

- Predict a duration d_i for each phoneme.
e.g. $D = [2, 2, 3, 1]$, indicating that the first phoneme lasts for 2 frames, the second for 2 frames, the third for 3 frames, and the fourth for 1 frame.
- Use aligned targets during training.
- At inference, predicted durations determine output length.

Length regulator

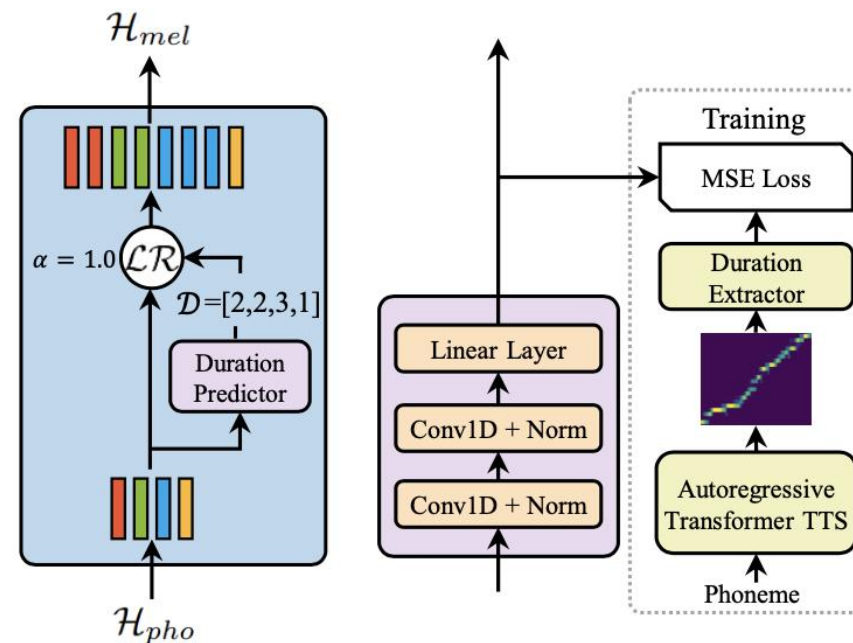
- A parameterless module that merely performs the "copy" operation.

e.g.

$h_{\text{phone}} = [a, b, c]$

$d = [2, 1, 3]$

$h_{\text{frame}} = [a, a, b, c, c, c]$



Code: Duration-based expansion

ming024/FastSpeech2 / model/modules.py

d4e79eb52e · L182-194

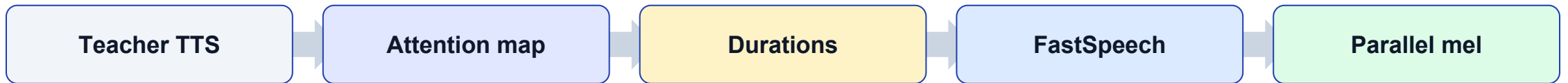
```
182 def expand(self, batch, predicted):
183     out = list()
184
185     for i, vec in enumerate(batch):
186         expand_size = predicted[i].item()
187         out.append(vec.expand(max(int(expand_size), 0), -1))
188     out = torch.cat(out, 0)
189
190     return out
191
192 def forward(self, x, duration, max_len):
193     output, mel_len = self.LR(x, duration, max_len)
194     return output, mel_len
```

Unchanged source excerpt · visual wrapping and common indentation offset only

One utterance [N,H] + durations [N] -> [sum(d),H]. Durations are frame counts, not seconds.

Chung-Ming Chien · ming024/FastSpeech2 · MIT · modules.py L182-194 · d4e79eb52e · independent implementation

FastSpeech : Training Paradigm & Trade-offs



Teacher-Student Distillation Mechanism

Offline Teacher Guidance

- A pre-trained Autoregressive Teacher supplies explicit alignment maps and target mel-spectrograms.

Student Learning (FastSpeech)

- FastSpeech learns deterministic duration prediction and feature mapping directly from the teacher.

Trade-off: Pros vs. Cons

Pros (Inference Gains)

- No dynamic attention computation at inference.
- Enables fully parallel, deterministic length expansion & ultra-fast synthesis.

Cons (Training Trade-offs):

- Complex two-stage training pipeline.
- Performance is upper-bounded by the quality of the Teacher's attention map.

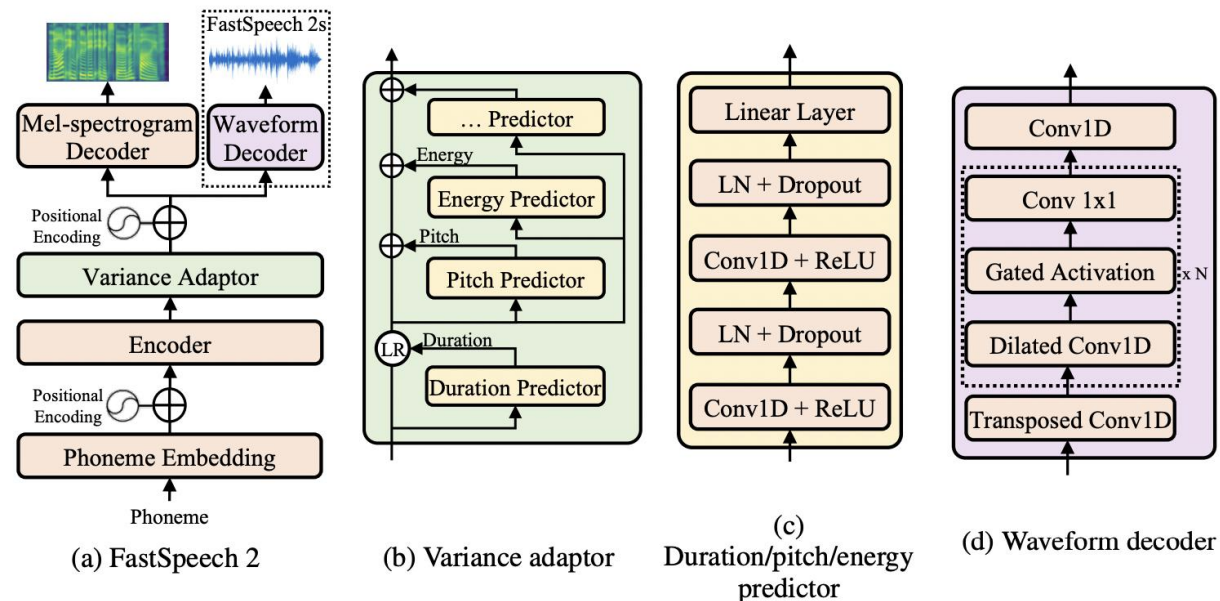
FastSpeech 2: Explicit Variance Adaptation

Motivation: Resolving One-to-many mapping

The same phoneme string can be spoken slowly, quickly, softly, loudly, or with different intonation.

1. Variance adaptor

- Duration controls rhythm.
- Pitch controls intonation and melody.
- Energy controls loudness and emphasis.



2. Duration/Pitch/Power Predictor

- Similar structure (consisting of several layers of 1D Conv)
- Sharing the output of the encoder, but each independently predicts three different numerical sequences

Code: Targets vs. Predicted Duration & Pitch Addition

ming024/FastSpeech2 / model/modules.py

d4e79eb52e · L128-143

```
128 if duration_target is not None:
129     x, mel_len = self.length_regulator(x, duration_target, max_len)
130     duration_rounded = duration_target
131 else:
132     duration_rounded = torch.clamp(
133         (torch.round(torch.exp(log_duration_prediction) - 1) * d_control),
134         min=0,
135     )
136     x, mel_len = self.length_regulator(x, duration_rounded, max_len)
137     mel_mask = get_mask_from_lengths(mel_len)
138
139 if self.pitch_feature_level == "frame_level":
140     pitch_prediction, pitch_embedding = self.get_pitch_embedding(
141         x, pitch_target, mel_mask, p_control
142     )
143     x = x + pitch_embedding
```

Unchanged source excerpt · visual wrapping and common indentation offset only

[B,N,H] -> [B,T,H]. Target or predicted duration expands x; frame-level pitch can then be added.

Chung-Ming Chien · ming024/FastSpeech2 · MIT · modules.py L128-143 · d4e79eb52e · independent F0-style code

Montreal Forced Aligner (MFA)

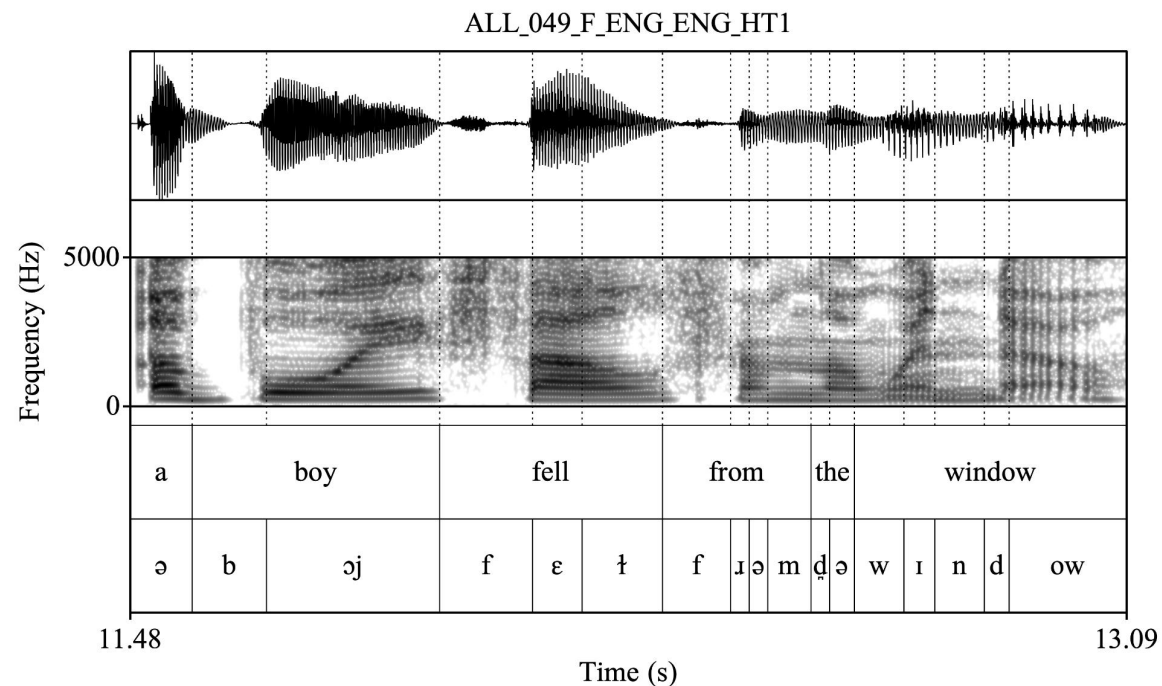
MFA Working Principle

Acoustic Feature Extraction

- Extract MFCC-like acoustic features.
- Use a GMM-HMM / pronunciation model.

Forced Alignment Search

- Search for the most probable state path under the transcript.
- Multi-pass optimization for speaker adaptation.



Review: alignment and controllability

Sequence length, module boundary, guarantees

1. Expand [a,b,c] with durations [1,3,2].
2. Is the length regulator after the entire variance adaptor?
3. Does a duration model guarantee that words are never skipped?

Reference answers: alignment

Explicit lengths improve control, not absolute correctness

1. Expand [a,b,c] with durations [1,3,2].

[a,b,b,b,c,c], giving six frames.

2. Is the length regulator after the entire variance adaptor?

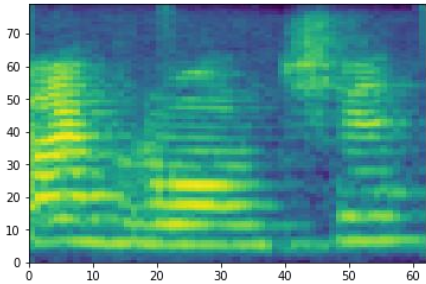
No. The length regulator is a component inside the variance adaptor.

3. Does a duration model guarantee that words are never skipped?

No. Frontend mistakes, duration errors, zero durations and acoustic failures can still reduce intelligibility.

Vocoder : Mel-Spectrogram to Waveform Synthesis

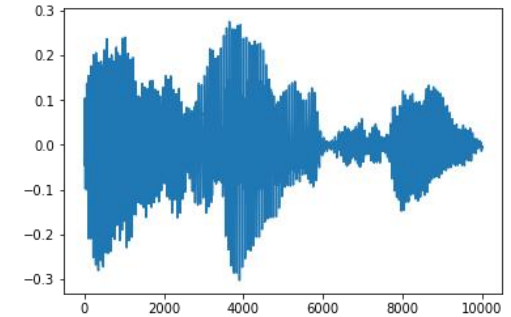
1. Task Pipeline:



Mel spectrogram



Vocoder



audio waveform

2. Core Question: What Must Be Reconstructed?

1. Phase Information & Fine Timing

- Mel-spectrograms throw away phase information; the vocoder must recover phase to generate natural time-domain signals.

3. Transient Consonants

- Accurately synthesizes high-frequency noise and unvoiced plosive/fricative sounds.

2. Harmonic Structure

- Restructures pitch periodicity, pitch harmonics, and clear fundamental frequencies

4. Natural Stochastic Details

- Fills in realistic background breathiness, environmental subtle noise, and acoustic texture.

WaveNet : Autoregressive Audio Generation

1. Probabilistic Modeling & Formula

(1) Autoregressive Formulation

Joint probability of audio waveform $\mathbf{x} = \{x_1, x_2, \dots, x_T\}$ is factorized into conditional probabilities:

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1})$$

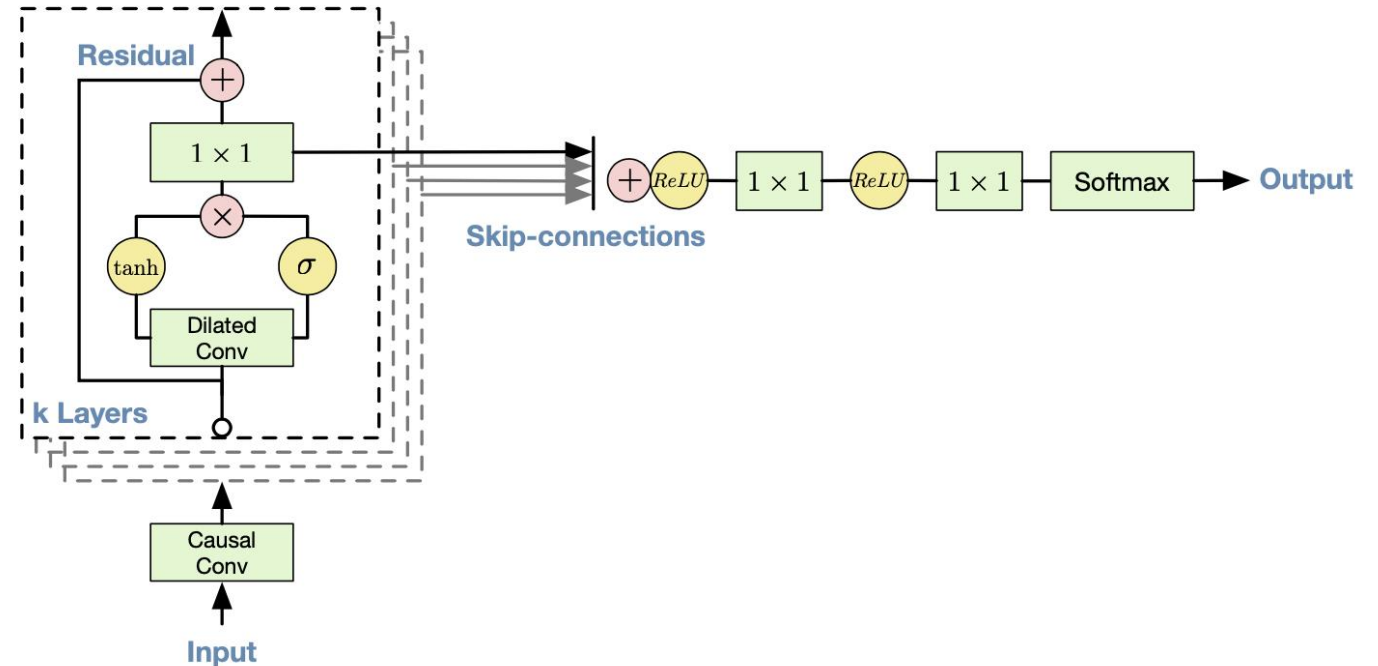
Each audio sample x_t is strictly conditioned on **all previously generated samples**.

(2) Quantized Categorical Output

Predicts a discrete probability distribution via Softmax:

$$p(x_t | x_1, \dots, x_{t-1}) \sim \text{Cat}(\pi_\theta)$$

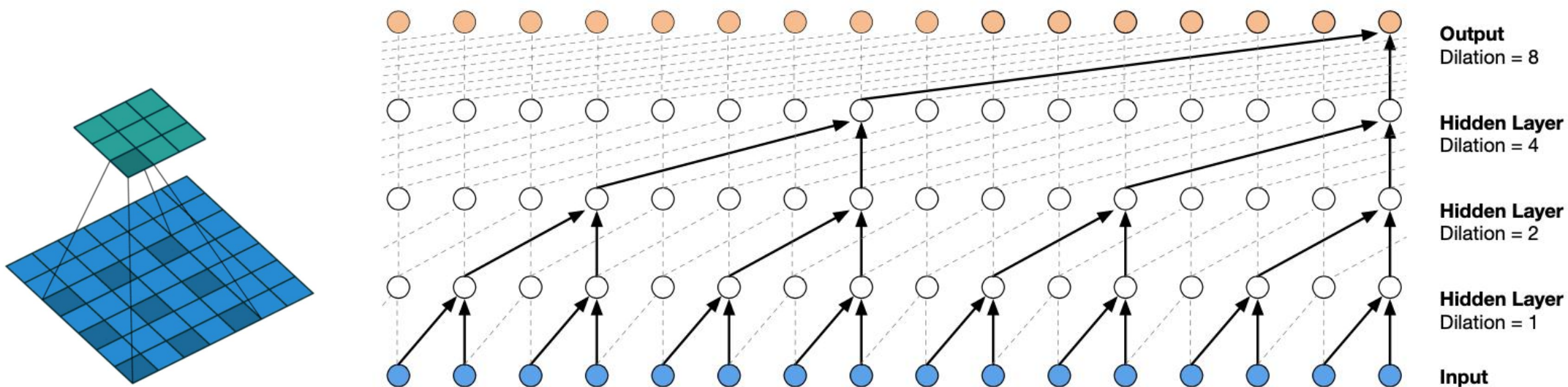
Quantizes 16-bit audio into 256 discrete values using mu-law companding algorithm.



2. Core Architectural Modules

- (1) Causal Convolutions
- (2) Dilated Convolutions
- (3) Gated Activation Units
- (4) Residual & Skip Connections

Dilated Causal Convolution : Expanding Receptive Field



Dual Mechanism Breakdown

Causality: Ensures prediction at step t depends only on samples at $t, t-1, t-2, \dots$, strictly avoiding future data leakage.

Dilated Rate Growth: Doubling dilation rates ($d = 1, 2, 4, 8, \dots$) skips input values with a step size, letting the filter apply over a larger area.

Receptive Field Formula

The effective receptive field R across L layers is computed as:

$$R = 1 + \sum_l (k_l - 1)d_l$$

Key Architectural Advantages

- Increase receptive field.
- Allow modeling long time dependencies.
- Causal for avoiding looking in the future.

Conditional WaveNet: Gated Activation Unit

Without acoustic conditioning

$$\mathbf{z} = \tanh(W_{f,k} * \mathbf{x}) \odot \sigma(W_{g,k} * \mathbf{x})$$

With local acoustic conditioning

$$\mathbf{z} = \tanh(W_{f,k} * \mathbf{x} + V_{f,k} * \mathbf{y}) \odot \sigma(W_{g,k} * \mathbf{x} + V_{g,k} * \mathbf{y})$$

Variable Notation

$\mathbf{x}$: Hidden states of past audio waveform history.

$\mathbf{y}$: Upsampled acoustic conditioning features.

$W_{f,k}, W_{g,k}$: Convolutional filter/gate weights for audio history.

$V_{f,k}, V_{g,k}$: Linear projection parameters for acoustic conditioning.

Dual-Branch Architecture

Filter Branch (tanh): Captures non-linear feature representations and acoustic transformations.

Gate Branch (sigma): Acts as a learnable element-wise memory gate ($0 \rightarrow 1$), regulating information flow.

μ -law Companding: Making Waveform Classification Tractable

Mathematical Definition

$$f(x_t) = \text{sign}(x_t) \frac{\ln(1 + \mu|x_t|)}{\ln(1 + \mu)}$$

(Where $\mu = 255$ to compress continuous amplitudes into 256 quantization bins).

Why Quantize & Compress?

Feasibility Challenge:

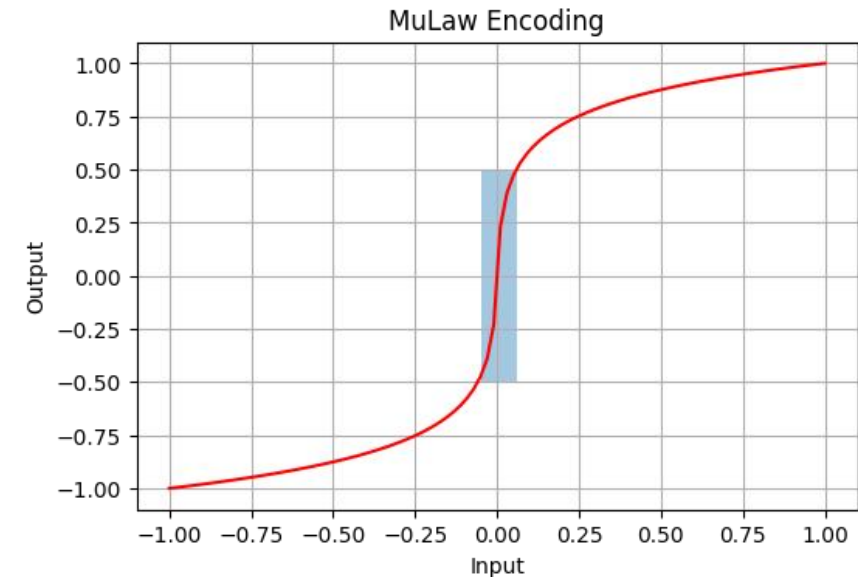
- Standard 16-bit PCM WAV audio has 65,536 possible amplitude values per sample.

Tractable Classification:

- Training a 256-way Softmax layer is computationally tractable and converges far easier than a 65,536-way Softmax.

Logarithmic Quantization:

- Reallocates dense quantization levels where human perception needs them most.



Perceptual Link — Psychoacoustic Motivation

High Resolution for Low Amplitudes:

- Human hearing is non-linear and far more sensitive to small amplitude variations in quiet sound segments.

Coarser Quantization for Loud Peaks:

- High-amplitude peaks mask quantization noise, allowing coarser binning without perceptual degradation.

Vocoder Evolution : Improving Quality, Eliminating Serial Bottlenecks

Generation	Model	Conditioning	Speed profile	Key idea
1	WaveNet	Past waveform	Very slow	Autoregressive waveform modeling
2	Conditional WaveNet	Past waveform + mel	Very slow	Content-controlled vocoding
3	Parallel WaveGAN	Noise + mel	Fast	GAN + multi-resolution STFT
4	MeIGAN / HiFi-GAN	Mel	Very fast	Multi-scale / multi-period discrimination

Core Transition: Shifted from Autoregressive (Sample-by-Sample) modeling to Non-Autoregressive GAN-based Parallel Generation.

Parallel WaveGAN : Non-Autoregressive Waveform Generation

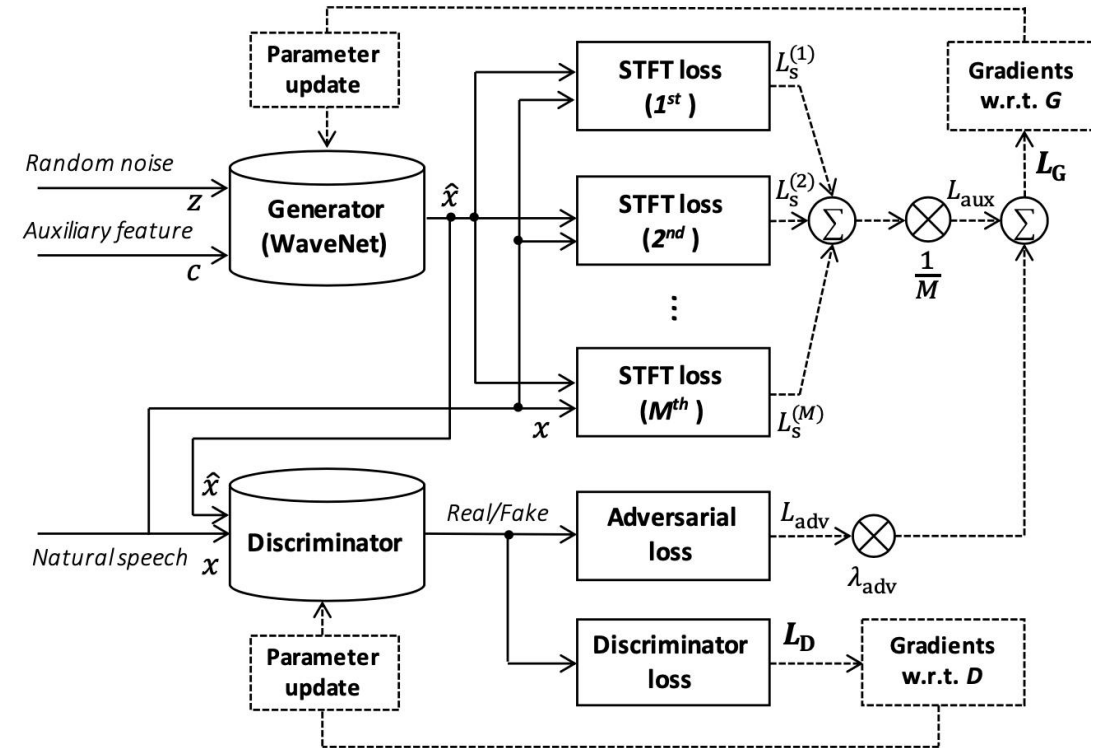
Primary Problem Solved: Traditional WaveNet suffers from severe serial bottlenecks.

1. WaveNet-based Non-Autoregressive Generator

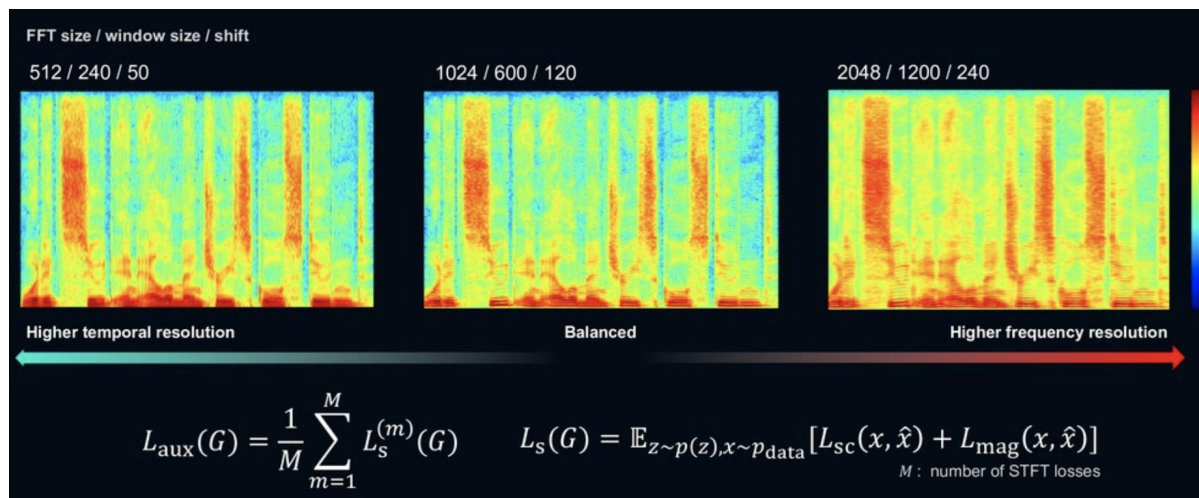
- Receives random noise and an auxiliary mel condition.
- Outputs the whole waveform in parallel.
- Uses a WaveNet-like stack without autoregressive sampling.

2. Additionally use Multi-STFT loss

- Remove the autoregressive mechanism
- Adopt GAN + STFT loss to ensure stable training and sound quality.



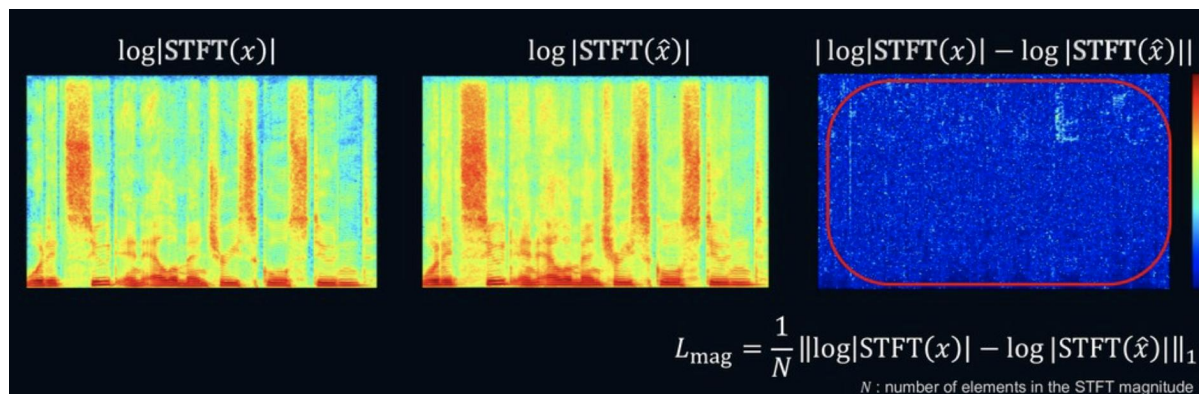
Multi-Resolution STFT Loss : Time-Frequency Balance



Several views of the same waveform

- **Short windows** emphasize temporal detail.
- **Long windows** emphasize frequency detail.
- Average both loss terms across resolutions.

$$L_{sc} = \frac{\|S - \hat{S}\|_F}{\|S\|_F}$$



Two complementary spectral errors

- Spectral convergence measures normalized overall Frobenius error.
- Log-magnitude distance emphasizes relative differences in low-amplitude regions.

Numerical clamps prevent log-zero; analysis settings must be disclosed.

Code : Multi-Resolution STFT Loss Aggregation

[kan-bayashi/ParallelWaveGAN / parallel_wavegan/losses/stft_loss.py](#)

86740373ec · L158-170

```
158 if len(x.shape) == 3:
159     x = x.view(-1, x.size(2)) # (B, C, T) -> (B x C, T)
160     y = y.view(-1, y.size(2)) # (B, C, T) -> (B x C, T)
161     sc_loss = 0.0
162     mag_loss = 0.0
163     for f in self.stft_losses:
164         sc_l, mag_l = f(x, y)
165         sc_loss += sc_l
166         mag_loss += mag_l
167     sc_loss /= len(self.stft_losses)
168     mag_loss /= len(self.stft_losses)
169
170     return sc_loss, mag_loss
```

Unchanged source excerpt · visual wrapping and common indentation offset only

Predicted and target audio -> two averaged scalars. Several analysis scales, not synthesized clips.

Tomoki Hayashi · [kan-bayashi/ParallelWaveGAN](#) · MIT · [stft_loss.py](#) L158-170 · 86740373ec · unofficial

MelGAN : Pure GAN-Based Non-Autoregressive Vocoder

Core Architectural Features

1. Fully Non-Autoregressive Synthesis

- Synthesizes audio in a single forward pass without autoregressive loops or sequential bottleneck.

2. End-to-End Upsampling Pipeline

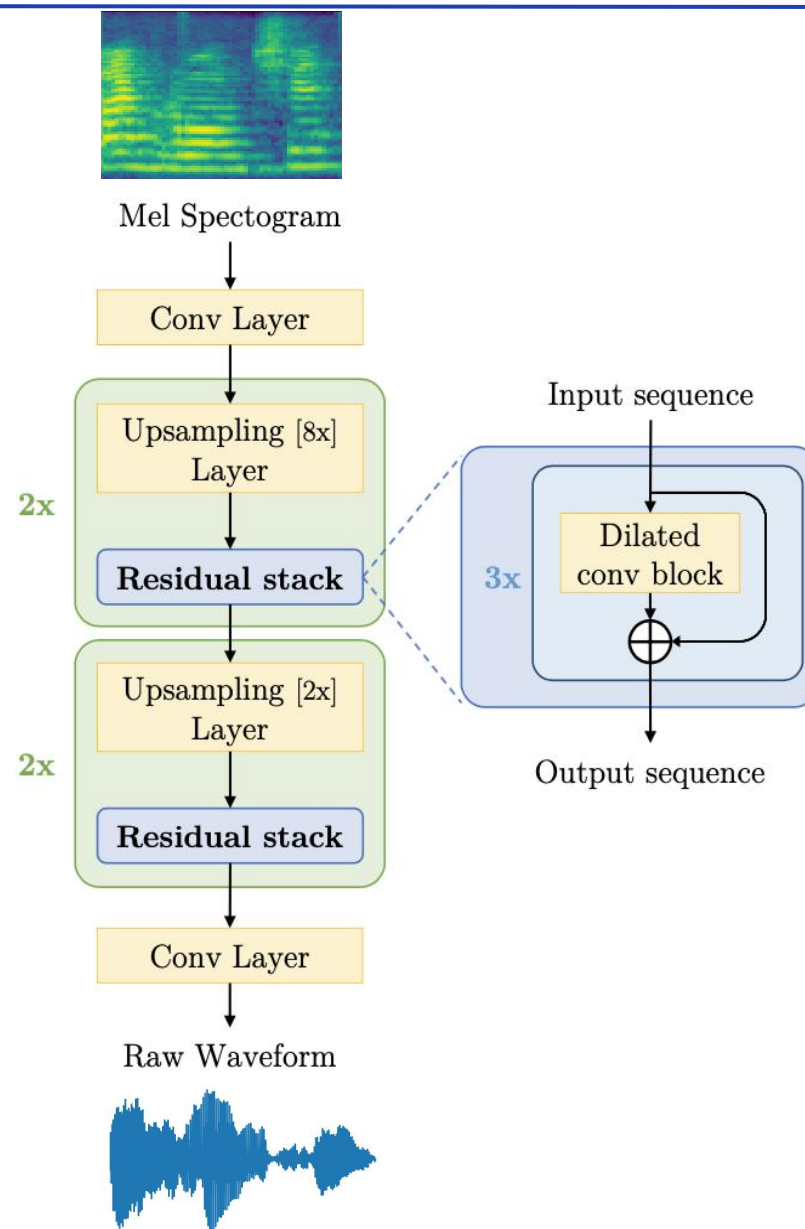
- Uses transposed convolutions (upsampling layers) to bridge the time-resolution gap between low-frequency Mel frames and high-frequency audio waveforms.

3. Receptive Field Expansion

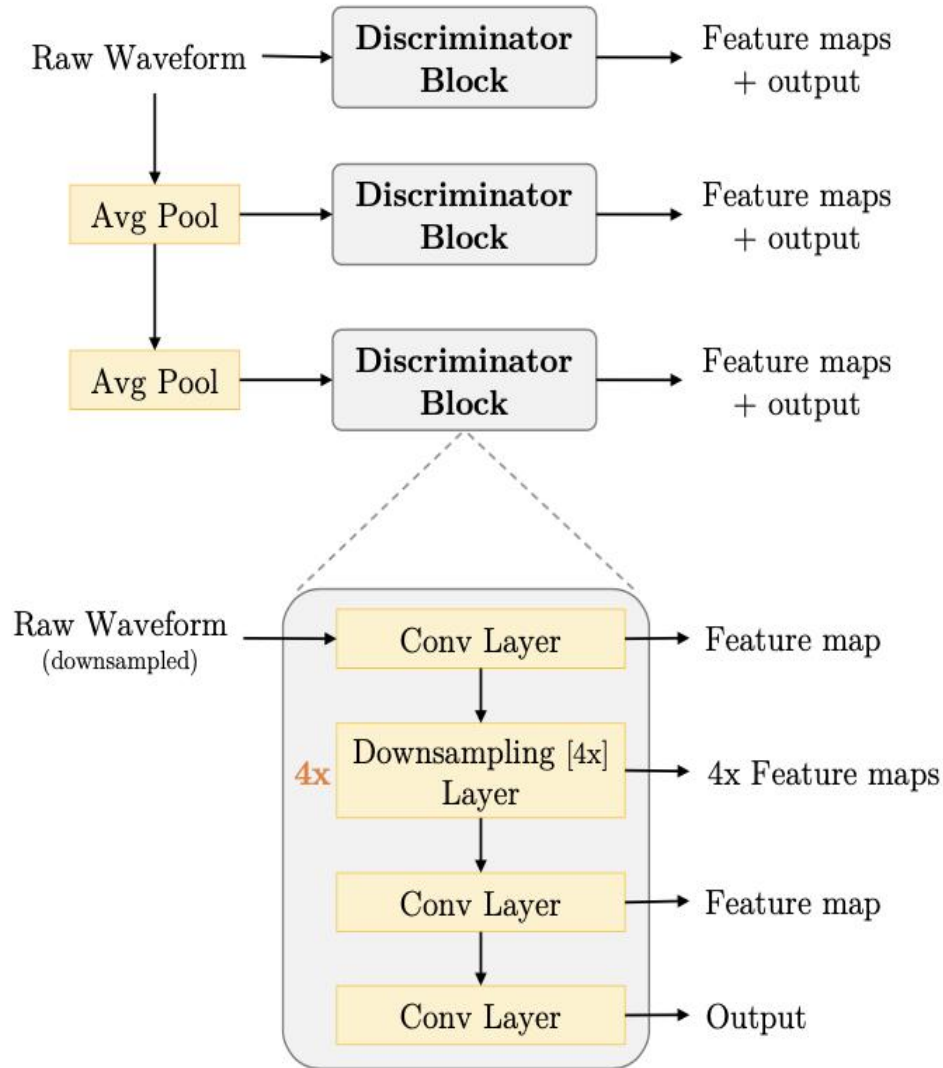
- Integrates Residual Stacks with Dilated Convolutions after upsampling to expand effective receptive fields and capture long-range temporal context.

4. No Distillation Required

- Directly trained end-to-end without needing teacher-student knowledge distillation or complex pre-training steps.



MelGAN : Multi-Scale Discriminator & Feature Matching



$$\mathcal{L}_{\text{FM}}(G, D_k) = \mathbb{E}_{x, s \sim p_{\text{data}}} \left[\sum_{i=1}^T \frac{1}{N_i} \|D_k^{(i)}(x) - D_k^{(i)}(G(s))\|_1 \right] \quad \min_{D_k} \mathbb{E}_x [\min(0, 1 - D_k(x))] + \mathbb{E}_{s, z} [\min(0, 1 + D_k(G(s, z)))] , \forall k = 1, 2, 3$$

$$\min_G \mathbb{E}_{s, z} \left[\sum_{k=1, 2, 3} -D_k(G(s, z)) \right]$$

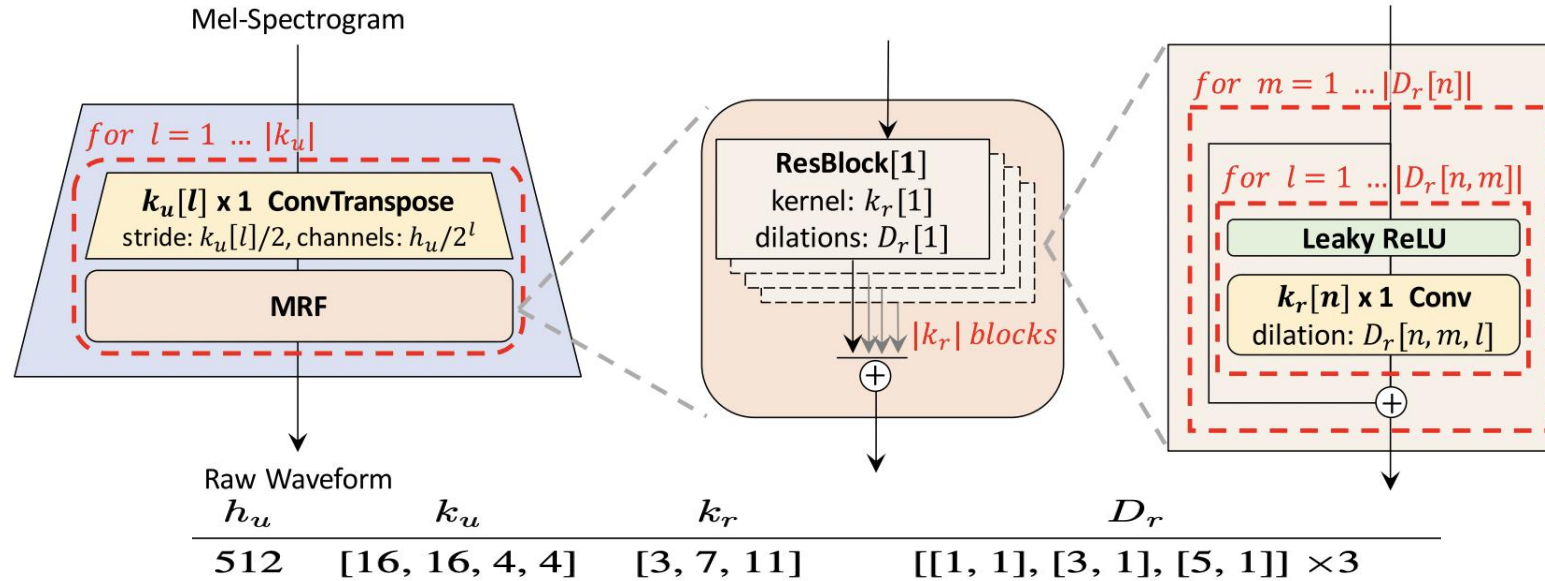
Multi-scale discriminator

- Different pooling scales inspect different temporal structure.
- The generator cannot win by matching only one resolution.

Feature matching

- Match intermediate discriminator features.
- Reduces mechanical artifacts and unstable training.

HiFi-GAN Generator : Multi-Receptive Field Fusion (MRF)



Overall Generator Pipeline & Innovation

Transposed Convolution Upsampling

- Progressive upsampling via $k_u = [16, 16, 4, 4]$ layers to match the target sample rate.

Multi-Receptive Field Integration (MRF)

- Instead of a single residual chain, HiFi-GAN processes features through multiple parallel ResBlocks with different kernel sizes (k_r) and dilation rates (D_r).
- Sums the outputs of all parallel ResBlocks to synthesize rich harmonic patterns.

MRF ResBlock Architecture Details

Hyperparameter Configuration:

- $h_u = 512$: Initial hidden feature dimension.
- k_u : kernel size of upsampling
- $k_r = [3, 7, 11]$: Parallel ResBlock kernel sizes (small, medium, large receptive fields).
- $D_r = [[1, 1], [3, 1], [5, 1]] \times 3$: Layered dilation rates expanding temporal fields without sparse holes.

Code: upsampling kernels and strides

[jik876/hifi-gan / models.py](#)

4769534d45 · L84-96

```
84 self.ups = nn.ModuleList()
85 for i, (u, k) in enumerate(zip(h.upsample_rates, h.upsample_kernel_sizes)):
86     self.ups.append(weight_norm(
87         ConvTranspose1d(h.upsample_initial_channel//(2**i), h.upsample_initial_channel//(2**
88             (i+1)),
89                         k, u, padding=(k-u)//2)))
90 self.resblocks = nn.ModuleList()
91 for i in range(len(self.ups)):
92     ch = h.upsample_initial_channel//(2**(i+1))
93     for j, (k, d) in enumerate(zip(h.resblock_kernel_sizes, h.resblock_dilation_sizes)):
94         self.resblocks.append(resblock(h, ch, k, d))
95
96 self.conv_post = weight_norm(Conv1d(ch, 1, 7, 1, padding=3))
```

Unchanged source excerpt · visual wrapping and common indentation offset only

Mel features -> longer hidden sequence. Stride sets expansion; kernel sets support; padding sets exact length.

Jungil Kong · [jik876/hifi-gan](#) · MIT · [models.py](#) L84-96 · [4769534d45](#) · author code

Code: multi-receptive-field fusion

[jik876/hifi-gan / models.py](#)

4769534d45 · L100-116

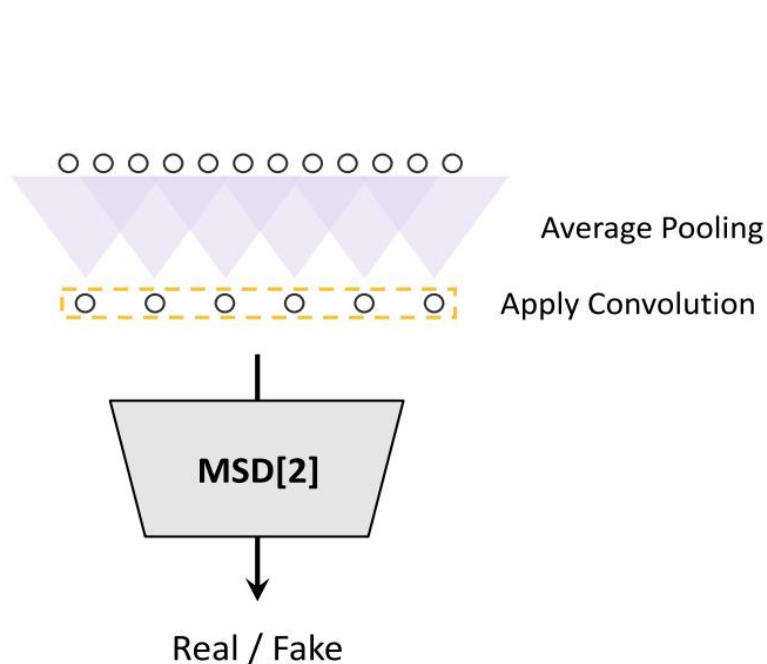
```
100 def forward(self, x):
101     x = self.conv_pre(x)
102     for i in range(self.num_upsamples):
103         x = F.leaky_relu(x, LRELU_SLOPE)
104         x = self.ups[i](x)
105         xs = None
106         for j in range(self.num_kernels):
107             if xs is None:
108                 xs = self.resblocks[i*self.num_kernels+j](x)
109             else:
110                 xs += self.resblocks[i*self.num_kernels+j](x)
111         x = xs / self.num_kernels
112     x = F.leaky_relu(x)
113     x = self.conv_post(x)
114     x = torch.tanh(x)
115
116     return x
```

Unchanged source excerpt · visual wrapping and common indentation offset only

Shared upsampled features -> averaged residual branches -> bounded mono waveform. Not mixed speech clips.

Jungil Kong · [jik876/hifi-gan](#) · MIT · [models.py](#) L100-116 · 4769534d45 · author code

Multi-Scale (MSD) vs. Multi-Period (MPD) Discriminators

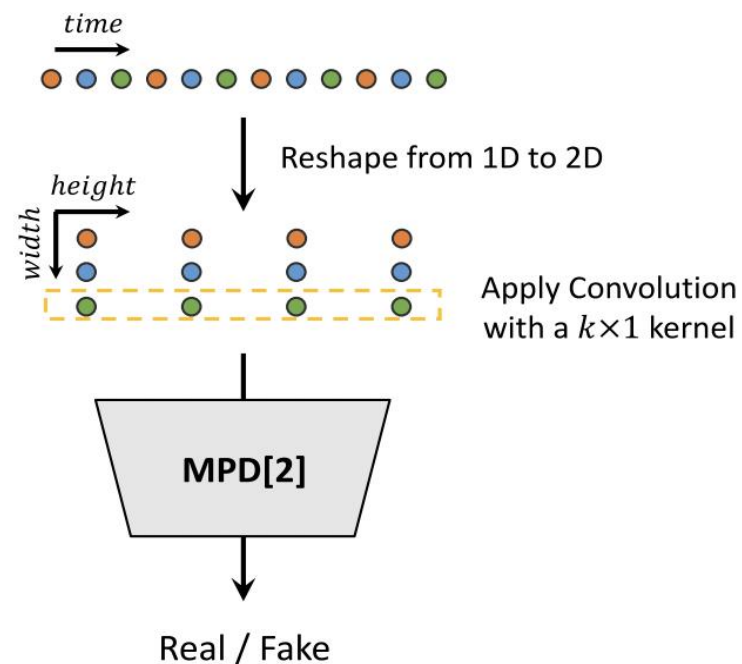


Temporal Resolution View

- Downsamples raw 1D audio via Average Pooling.
- Uses 1D Conv to inspect consecutive audio samples at coarse, medium, and fine scales.

Primary Objective

- Evaluates temporal smoothness, long-term trends, and phase consistency across raw waveforms.



Harmonic Pitch Periodicity View

- Folds 1D audio into 2D tensor of shape $[T/p, p]$.
- Uses 2D Conv with $k \times 1$ kernels along the period dimension p .

Primary Objective

- Captures pitch, fundamental frequencies (F0), and disjoint harmonic structures associated with period p .

Code: folding samples by period

[jik876/hifi-gan / models.py](#)

4769534d45 · L142-156

```
142 def forward(self, x):
143     fmap = []
144
145     # 1d to 2d
146     b, c, t = x.shape
147     if t % self.period != 0: # pad first
148         n_pad = self.period - (t % self.period)
149         x = F.pad(x, (0, n_pad), "reflect")
150         t = t + n_pad
151     x = x.view(b, c, t // self.period, self.period)
152
153     for l in self.convs:
154         x = l(x)
155         x = F.leaky_relu(x, LRELU_SLOPE)
156         fmap.append(x)
```

Unchanged source excerpt · visual wrapping and common indentation offset only

[B, 1, T] -> padded [B, 1, T/p, p]. For p = 3: [x0,x1,x2], [x3,x4,x5]. Fixed periods, not pitch labels.

Jungil Kong · [jik876/hifi-gan](#) · MIT · [models.py](#) L142-156 · [4769534d45](#) · author code

HiFi-GAN : Criteria

$$L_G = L_{adv} + \lambda_{mel} L_{mel} + \lambda_{fm} L_{fm}$$

1. Least Squares Adversarial Loss (L_{adv})

Real scores approach 1; generated scores approach 0.

Generator

Generated scores approach 1.

$$\mathcal{L}_{Adv}(D; G) = \mathbb{E}_{(x,s)} [(D(x) - 1)^2 + (D(G(s)))^2]$$

$$\mathcal{L}_{Adv}(G; D) = \mathbb{E}_s [(D(G(s)) - 1)^2]$$

2. Mel-Spectrogram Loss (L_{Mel})

Compare target and generated spectral features.

3. Feature Matching Loss (L_{fm})

Compare intermediate discriminator activations.

Code : Discriminator Feature Matching Loss

[jik876/hifi-gan / models.py](#)

4769534d45 · L251-257

```
251 def feature_loss(fmap_r, fmap_g):
252     loss = 0
253     for dr, dg in zip(fmap_r, fmap_g):
254         for rl, gl in zip(dr, dg):
255             loss += torch.mean(torch.abs(rl - gl))
256
257     return loss*2
```

Unchanged source excerpt · visual wrapping and common indentation offset only

Real / generated layer activations -> summed mean absolute differences. D update control is outside this loss.

Jungil Kong · jik876/hifi-gan · MIT · models.py L251-257 · 4769534d45 · author code

Weight Normalization in Vocoders

Core Reparameterization Formula

$$\mathbf{w} = \frac{g}{\|\mathbf{v}\|} \mathbf{v}$$

Vector & Scale Decoupling

- $\mathbf{v}$: Unconstrained vector determining the filter's direction (scaled to unit length).
- g : Scalar parameter controlling weight scale.

Optimization Benefits

- Accelerates gradient convergence and stabilizes deep neural networks without batch dependency.

Why Not Batch Normalization (BN)?

- BN depends on running batch statistics, which fails with dynamic audio lengths and small batch sizes.
- WN normalizes layer parameters directly, avoiding activation distortion across audio time steps.

Zero-Cost Inference via Weight Folding

- Post-training: Fuses g and $\mathbf{v}$ into a static weight $\mathbf{w}$.
- Removes WN wrapper to eliminate runtime overhead during deployment.

Review: vocoder mechanisms

1. Why is mel not uniquely invertible?
2. Are kernels [16, 16, 4, 4] a 4096x expansion?
3. In MPD [T/p, p], which axis indexes phase?
4. Why use more than one STFT window?

Check information loss, stride versus kernel, reshaped axes and analysis resolution.

Reference answers: vocoders

1. Mel features discard information.

Band aggregation and phase loss prevent a unique waveform inverse.

2. No: use the expansion rates, not kernel sizes.

For HiFi-GAN V1, [8, 8, 2, 2] yields 256x expansion.

3. The p-sized column axis indexes phase positions.

The other axis indexes successive groups of consecutive p samples.

4. Different windows capture complementary detail.

Short windows emphasize time; long windows resolve frequency.

Correct units and axes matter as much as architecture names.

How to Measure Quality?

Dimension	Primary Listener Question	Typical failure
Intelligibility	Can I understand the words?	Skipped or blurred phonemes
Naturalness	Does it sound like a person?	Robotic prosody or buzz
Similarity	Does it sound like the target speaker?	Wrong timbre / identity
Pleasantness	Is it comfortable to listen to?	Harshness, hiss, fatigue
Prosody	Are intonation and pauses appropriate?	Flat or misplaced emphasis

Mean Opinion Score (MOS)

MOS	Quality	Impairment
5	Excellent	Imperceptible
4	Good	Perceptible but not annoying
3	Fair	Slightly annoying
2	Poor	Annoying
1	Bad	Very annoying

Standard Protocol Requirements

- Requires controlled environments & listener panels.
- Must report sample size (N) and uncertainty.

Mathematical Formula

$$MOS = \frac{\sum_{n=1}^N R_n}{N}$$

Key Strengths

- Intuitive and widely understood across academic Intuitive and widely understood across academic

Limitations & Pain Points

- Expensive & Time-Consuming: High human labor cost.
- High Variance & Ceiling Effect: Hard to resolvesubtle model improvements near human quality.

MUSHRA & Side-by-side (AB) Tests

MUSHRA

- Evaluates multiple stimuli simultaneously against:
 - Explicit Reference
 - Hidden Reference & Low-Quality Anchor
- Uses a continuous 0–100 score scale.

Best Used For

- Differentiating between multiple high-quality systems where MOS encounters ceiling effects.

Decision Rule Guidelines:

- *Use MUSHRA when you need to rank and benchmark a set of multiple systems.*
- *Use AB Testing when asking a focused A/B preference question for a single model change.*

Side-by-side / AB

- Directly compares two audio samples (A vs B).
- Task: Listener selects the preferred option.

Key Advantages

- Binary Choice: Eliminates individual differences in rating-scale habits across listeners.
- High Sensitivity: Extremely sensitive to hard, audible errors and artifacts.

Summary & Metric Family Matrix

Ask one condition

Noise Check:

“Does the sample contain audible background noise?”

Intelligibility Check:

“Can the listener identify the sentence?”

Speaker Identity Check:

“Does the voice sound like the target speaker?”

Metric family	Output	Best use
MOS	1–5 average	Overall quality
MUSHRA	0–100 per sample	Fine-grained ranking
AB / SBS	A or B	Preference for a change
Binary	Yes / No	Failure diagnosis

Purpose

Converts vague listener feedback into actionable engineering diagnostics (e.g., failure tagging)

The evaluation protocol should be designed around the failure you want to detect.

Crowdsourcing & Statistical Rigor

Crowdsourcing Implementation & Quality Control

Crowdsourcing Advantages

- **Scalability:** Access to a large, diverse listener pool across demographics and locations.
- **Cost Efficiency:** Significantly cheaper than setting up dedicated acoustic laboratories.

Essential Quality Controls

- Pre-test Screening (hearing checks).
- Device Checks (enforcing headphone usage).
- Attention Trap Checks (filtering random clickers).

Confidence interval

Mathematical Formula

$$CI = \bar{x} \pm z \frac{s}{\sqrt{n}}$$

Reporting Requirements & Best Practices

- **Complete Transparency:** Always report mean score, sample size (n), and CI together

Critical Statistical Insight

- A **0.05** MOS difference is not automatically meaningful unless supported by CI bounds.

Public Datasets in TTS

Dataset	Typical use	Coverage	Main caveat
LJSpeech	Single-speaker baseline	Clean English, ~24 h	One speaker / narrow style
LibriTTS	Multi-speaker and robustness	Many speakers, audiobook speech	Variable recording conditions
Common Voice	Multilingual / accents	Crowdsourced languages	Uneven quality and labels
OpenTTS / analogs	Supplementary resources	Task- or language-specific	License and comparability

Core Takeaway: A model trained on one clean single-speaker dataset should never be described as universally robust without explicit zero-shot or out-of-domain evaluation.

Selected references

- Tacotron 2 — arXiv:1712.05884
- Guided Attention — arXiv:1906.00672
- Global Style Tokens — arXiv:1803.09017
- FastSpeech / FastSpeech 2 — arXiv:1905.09263, arXiv:2006.04558
- WaveNet — arXiv:1609.03499
- Parallel WaveGAN — arXiv:1910.11480
- MelGAN — arXiv:1910.06711
- HiFi-GAN — arXiv:2010.05646
- Weight Normalization — arXiv:1602.07868
- Montreal Forced Aligner — Interspeech 2017

All architecture figures in this deck are adapted from the user-provided reference PPTX; the visual system follows the previously supplied source-separation courseware.